دليل داخلي · للمشاريع القادمة

منهجية العمل
الهوية البصرية، الستراكشر، والفلاتر

كيف أبني الهوية، وكيف أقسّم مشاريع الداشبورد والتطبيقات، وما القرارات الثابتة والباكجات التي أعتمدها. الدليل عام لأي مشروع، ويُستبدل فيه `<brand>` باسم المشروع.

1. القرارات الثابتة
2. تقسيمة المنظومة (الريبو)
3. الهوية البصرية: المنهجية
4. حزمة التصميم المحلية
5. ستراكشر مشروع Flutter
6. الطبقات: من الـ API إلى الشاشة
7. إدارة الحالة بلا setState
8. الشبكة والأخطاء
9. الكاش والعمل دون اتصال
10. الراوتر والتنقل
11. الداشبورد (ويب)
12. تطبيقات الموبايل
13. التحميل والحالات الأربع
14. النماذج والتحقق
15. الباكجات المعتمدة
16. التسمية والكود والـ Lint
17. قائمة البدء: مشروع جديد / ميزة جديدة

1. القرارات الثابتة

- **لا `setState` بأي مكان بالمشروع.** كل حالة (حتى إظهار كلمة السر أو التبويب المحدد) بـ Bloc أو Cubit. الحالة المحلية البسيطة بـ `ToggleCubit` / `ValueCubit<T>`.
- **لا ملف فوق 300 سطر.** الملف الكبير يُقسَّم: ويدجتس الشاشة لـ `widgets/`، الأحداث والحالة بملفات `part`، والمسارات لملفات حسب المجال.
- **إدارة الحالة Bloc/Cubit فقط** (`flutter_bloc`)، ولا Provider ولا GetX ولا Riverpod.
- **لا قيمة تصميم مكتوبة بالواجهة:** لا `Color(0x...)`، لا `TextStyle` من الصفر، لا أرقام مسافات عشوائية، لا `Duration(milliseconds: 300)`. كلها من التوكنز (`AppMotion`, `AppRadius`, `AppSpace`).
- **لا مسار مكتوب نصاً داخل شاشة:** كل المسارات بـ `Routes`.
- **كل شاشة بأربع حالات:** تحميل بهياكل (Shimmer)، خطأ مع إعادة محاولة، فارغة كدعوة، ومحتوى.
- **لا استثناء يصل للواجهة:** كل طلب يرجع `ApiResult` (نجاح أو `Failure`).
- **الواجهة تتصرف حسب نوع الخطأ** (`FailureType`) وليس نص الرسالة.
- **كل التعديلات من مكان واحد:** `AppForm` / `FormDialog` فوق `AppFormCubit` (تحميل + أخطاء حقول 422 موحّدة).
- **الميزات تستورد من core ملفاً واحداً:** (barrel) `core/core.dart`.
- **عربي فقط، RTL، ثيمان (فاتح وداكن)،** الأرقام لاتينية، والأرقام والمبالغ والهواتف باتجاه LTR.
- **"تقليل الحركة" يُحترم دائماً:** كل حركة تفحص `AppMotion.reduced(context)`.
- **لا overflow بأي مقاس:** تكبير خط النظام محصور بحد (`MediaQuery.withClampedTextScaling`).

2. تقسيمة المنظومة (الريبو)

الواجهات كلها بريبو واحد، والباك اند بريبو منفصل. كل واجهة مشروع Flutter مستقل، ومجلد الهوية بجانبها:

```
<brand>-frontend/
├── README.md                  جدول: المجلد ← المشروع
├── design/                    الهوية البصرية (القسم 3)
├── super-admin-dashboard/     داشبورد صاحب المنصة (Flutter Web)
├── tenant-dashboard/          داشبورد العميل/المؤسسة (Flutter Web)
├── manager-mobile-app/        تطبيق حسب الدور (Android / iOS)
├── staff-mobile-app/          تطبيق دور آخر (عدة أدوار بتطبيق واحد إن تشابهت)
└── end-user-mobile-app/       تطبيق المستخدم النهائي
```

- **تطبيق لكل جمهور** (وليس لكل دور): الأدوار المتقاربة (مثلاً مدير ومشرف) بتطبيق واحد يتبدّل محتواه حسب الدور.
- **كل المشاريع بنفس الستراكشر ونفس core** تقريباً، حتى تنتقل الميزة أو الويدجت بين المشاريع بالنسخ.
- **كل مشروع فيه نسخة من حزمة التصميم** (path dependency) بـ `packages/<brand>_design`.

- اسم المجلدات بالإنجليزية بلا مسافات قدر الإمكان: أدوات Flutter تتعطل أحياناً مع المسارات العربية.
- كل مشروع له README قصير: ما هو، الخادم الذي يكلّمه، أمر التشغيل والبناء، والستراكشر والقواعد.

3. الهوية البصرية: المنهجية

3.1 مجلد الهوية

```
design/brand-identity/
├── README.md                "ابدأ من هنا": من أنت ← ماذا تفتح
├── brand-guide.html         دليل تفاعلي يُفتح بالمتصفح (حركات حيّة + زر الثيم الداكن)
├── docs/identity-spec.md    التوصيف الكامل: الألوان، الخطوط، الزخارف، الحركة، التفاعلات، القوالب، إعداد Flutter
├── tokens/tokens.json       المصدر الوحيد للقيم (ألوان بالثيمين، خطوط، مسافات، زوايا، ظلال، مقاسات، حركة)
├── logo/svg · png · app-icons    أيقونات المتاجر + أيقونة، عرضي، طولي، ختم، مفرّغ (فاتح وداكن) و favicon
├── ornaments/svg            الزخارف: currentColor فاتح، داكن، وأحادي
├── fonts/                   الخطوط مع رخصتها (OFL)
├── flutter/<brand>_design/  حزمة Flutter + تطبيق عرض example/
├── screenshots/             لقطات تطبيق العرض
└── (اختياري) social/ print/ video/ email/ studio/ tools/   البراند الممتد
```

3.2 الفكرة قبل الألوان

أبدأ بجملة تلخص الشخصية (الإلهام، الإحساس، القاعدة الذهبية)، ثم **أربع قواعد** تحكم كل قرار تصميم:

1. **الزخرفة بمكانها فقط:** للحظات الخاصة (الدخول والبدء، بانر الرئيسية، رأس التطبيق، الحالات الفارغة، النجاح). شاشات العمل اليومي (القائمة، الجداول، الفورمات) نظيفة تماماً. مكان مزخرف واحد بالشاشة على الأكثر.
2. **لون واحد لكل معنى:** لون الفعل = "اضغط هنا"، لون اللمعة = "المحدد/المميز"، لون الهيكل = القائمة والبانر، وألوان الحالات للحالة فقط.
3. **كل شيء يتحرك، ببطء:** الحركة الدائمة بطيئة جداً، والتفاعلية سريعة وواضحة.
4. **"تقليل الحركة" مبني بكل مكوّن.**

3.3 الألوان بالأدوار (وليس بالأسماء)

الدور	الاستخدام
`bg`	أرضية الصفحات
`surface` / `surfaceAlt`	البطاقات / ترويسة الجدول والمرور
`border` / `borderStrong`	الحدود
`ink` / `inkMuted` / `inkSubtle`	النص بثلاث درجات
`hero` / `sidebar` (لون الهيكل)	القائمة، البانر، شاشة الدخول
`primary` + `Hover` / `Soft` / `Ink` / `onPrimary`	الفعل: الزر الأساسي، أعمدة الرسوم

`accent` + `Soft` / `Ink`	اللمعة: العنصر المحدد، حد البطاقة عند المرور
`spark` (لون ثالث صغير)	نقطة الإشعار، قلب الفاصل
`success / warning / danger / info`	كل حالة بثلاث درجات: قوي، هادئ (خلفية)، حبر (نص)

- كل لون بقيمتين (فاتح وداكن) ضمن `Palette.light` و `Palette.dark`.
- لون الخطر مختلف بوضوح عن لون الفعل (لا يتشابهان).
- الشارة دائماً: خلفية هادئة + نص بالحبر. اللون وحده لا يكفي لنقل معنى.
- **زر أساسي واحد بكل شاشة.**
- تباين 4.5:1 للنص بالثيمين، و 3:1 للنص الكبير والأيقونات.
- ما يدخل بـ `ColorScheme` يُقرأ بـ `context.scheme`، والباقي بامتداد ثيم `context.colors`.

3.4 الخطوط

- **خطان فقط:** خط واجهة متغيّر (variable) لكل النصوص، وخط عرض (display) للتحيات وشاشة الدخول وعناوين الحالات الفارغة والنجاح. خط العرض لا يُستخدم لعناوين الصفحات ولا الأزرار ولا الجداول.
- الخطوط **مضمّنة بالحزمة** ولا تُحمَّل من الإنترنت.
- سلّم أنماط ثابت (`TType`): displayXl, displayMd, h1, h2, h3, stat, bodyLg (نص الموبايل), body, label, caption. أي نمط جديد يُشتق بـ `copyWith`.
- الخط المتغيّر: الوزن عبر `fontVariations: [FontVariation('wght', 600)]` مع `fontWeight`.

3.5 المسافات والأشكال والظلال

- شبكة 4px + فراغات جاهزة: `s1=4, s2=8, s3=12, s4=16, s5=20, s6=24, s8=32, s10=40, s12=48` — `gap1..gap8`.
- زوايا: `sm 8 · md 12 (الحقول) · lg 16 · xl 22 · full 999 (الأزرار والشارات والبحث)`.
- شكل توقيعي للبطاقة (مثلاً زوايا علوية أعرض من السفلية) يتكرر بكل البطاقات، وشكل خاص للبانر والنوافذ.
- البطاقة بلا ظل وهي ساكنة، ظل خفيف عند المرور، وظل عائم للنوافذ وبطاقة الدخول فقط.
- مقاسات ثابتة: عناصر التحكم 34/42/50، صف الجدول 52، الشريط العلوي 64، القائمة 272 ومطوية 80، أقصى عرض للمحتوى 1320.

3.6 الحركة

التوكن	القيمة	الاستخدام
`instant`	100ms	الضغط (تصغير 0.97)
`fast`	180ms	المرور، انزلاق عنصر القائمة
`normal`	260ms	ارتفاع البطاقة، تبديل حالة القسم
`slow`	420ms	تبديل الصفحة، طي القائمة
`entrance`	600ms	دخول العناصر (+70ms لكل عنصر)

`curve`	Cubic(0.2, 0.8, 0.2, 1)	المنحنى الأساسي
`emphasized`	Cubic(0.2, 0, 0, 1)	التحولات الكبيرة

- لا ارتداد ولا اهتزاز. لا حركة فوق 1.2 ثانية إلا الحركات الدائمة البطيئة (زحف نقش 40s، دوران 70s...).
- حركات جاهزة كمكونات: `Entrance(index:)` ، `CountUp` للأرقام، طلوع أعمدة الرسوم، ، `HoverLift` ، `Shine` ، `PressScale` على الزر الأساسي، `PingDot` للإشعار.
- مع "تقليل الحركة": العدّ والأعمدة والدخول تظهر بقيمتها النهائية فوراً، والدائمة تقف.

3.7 التفاعلات

الداشبورد (ماوس): مرور + ضغط + تركيز لوحة مفاتيح (حلقة 2px بلون اللمعة) + محدد + معطّل (شفافية 45%) لكل عنصر: الأزرار، البطاقات، عنصر القائمة (انزلاق 3px + مؤشر)، صف الجدول، عمود الرسم (فقاعة القيمة)، الحقل (حد 2px بلون الفعل).

التطبيقات (لمس): لا مرور. تصغير 0.97 لحظة الضغط على كل ما يُضغط،
`HapticFeedback.selectionClick()` للأفعال المهمة وتبديل التبويب، أهداف ضغط 48px على الأقل.

3.8 اللغة (Microcopy)

- عربية فصحى مبسطة. الأزرار فعل مباشر: "إضافة"، "تسجيل دفعة"، "حفظ".
- الخطأ: ماذا حدث ثم ماذا تفعل. "الاسم مستخدم مسبقاً. جرّب اسماً آخر."
- النجاح قصير بصيغة الماضي: "تم الحفظ."
- الحالة الفارغة دعوة لا اعتذار: "لا توجد عناصر بعد. أضف أول عنصر."
- التحية تحسب الوقت: صباح الخير / مساء الخير.

3.9 قوالب الشاشات

الداشبورد: ≥1024 قائمة مفتوحة 272 + شريط علوي 64 + محتوى بأقصى عرض 1320 وهوامش 24. من 600 إلى 1024 قائمة مطوية 80 (أيقونات + تلميحات). <600 القائمة درج من زر، البحث يختفي، والبطاقات عمود واحد.
شبكة البطاقات: الأعمدة = العرض ÷ 230 بفراغ 12.

الموبايل: رأس بلون الهيكل بزخرفة وتحية (المكان المزخرف الوحيد)، إجراءات سريعة (3 بطاقات)، بطاقات محتوى نظيفة، وشريط تنقل سفلي بأربعة عناصر.

4. حزمة التصميم المحلية

```
packages/<brand>_design/
├── pubspec.yaml        flutter_svg تعريف الخطوط + الأصول + فقط
├── assets/logo/  assets/fonts/
└── lib/
    ├── <brand>_design.dart     barrel: export src/*
    └── src/
        ├── tokens.dart    القيم الخام (Palette.light / dark) - لا تُستخدم مباشرة بالواجهات
        ├── theme.dart     ThemeData فاتح وداكن + ThemeExtension للألوان الإضافية
        ├── motion.dart    Entrance, CountUp, HoverLift, PressScale, Shine ...
        ├── ornaments.dart الزخارف بـ CustomPainter (تتلون بالثيم وتقف مع تقليل الحركة)
        └── widgets.dart   مكونات الهوية (بطاقة، شارة، فاصل، EmptyState ...)
```

```
dependencies:
  <brand>_design:
    path: packages/<brand>_design

// خط من داخل الحزمة يُشار له بـ packages/<اسم الحزمة>/<العائلة>
static const sans = 'packages/<brand>_design/MainFont';
```

- داخل كل مشروع، طبقة `core/theme/` رقيقة فوق الحزمة: `app_colors.dart`، `app_theme.dart` (امتداد `context.colors`)، `app_tokens.dart` (`AppFonts`، `AppSpace`، `AppRadius`، `AppSize`، `AppMotion`)، `theme_cubit.dart` (يتبع الجهاز حتى يختار المستخدم ويُحفظ اختياره)، و `system_ui.dart` (شريطا النظام شفافان).
- أيقونة التطبيق بـ `flutter_launcher_icons`، وشاشة البدء الأصلية بـ `flutter_native_splash` (لون الهيكل + الشعار، ونسخة داكنة).

5. ستراكشر مشروع Flutter

```
lib/
├── main.dart          تهيئة: GlobalErrorGuard، DI، (الموبايل: edge-to-edge، الإشعارات، الاتجاه)
ثم runApp
├── app.dart           MaterialApp.router: الثيمان، العربية، RTL، BlocProviders العامة، builder
العام
├── core/
│   ├── core.dart      barrel: الميزات تستورد هذا فقط
│   ├── bloc/          Slice، LoadCubit، ListCubit، ValueCubit/ToggleCubit، AppFormCubit،
pullRefresh
│   ├── config/        app_config.dart: رابط الـ API (dart-define)، المهلات، حدود تكبير الخط،
العنوان
│   ├── di/            injection.dart (التخزين والشبكة) + feature_injection.dart (الميزات)
│   ├── error/         failure، error_handler، validation_translator، global_error_guard
│   ├── layout/        هيكل ما بعد الدخول (AppShell+Sidebar+Topbar أو MobileShell+TabBar)
│   ├── models/        موديلات مشتركة صغيرة (مرجع: id + name)
│   ├── networking/    dio_factory، api_client، api_result، auth_interceptor، list_query، paged
│   ├── network/       (موبايل) connection_cubit، connectivity_interceptor،
offline_cache_interceptor
│   ├── push/          (موبايل) push_service، push_permission
│   ├── responsive/    Breakpoints، context.isMobile/responsive()، ResponsiveGrid
│   ├── router/        app_router، routes (الثوابت)، nav (open/back)، ملفات مسارات حسب المجال
│   ├── storage/       session_storage، (موبايل) response_cache + cache_store (io/web)
│   ├── theme/         طبقة الثيم فوق حزمة التصميم
│   ├── utils/         formatters، json_x، labels، validators، debouncer، launcher، file_pick،
blob_opener
│   └── widgets/       كل الويدجتس المشتركة (القسم 13 و 14) + charts/
└── features/<feature>/
    ├── domain/models/<feature>_models.dart    كل موديلات الميزة بملف واحد (يُقسّم إن كبر)
    ├── data/repo/<feature>_repository.dart    العقد: abstract interface class (دوال بلا جسم)
    ├── data/datasources/<feature>_remote_datasource.dart   التنفيذ: الطلبات من الباك اند
    └── presentation/
        ├── bloc/      <feature>_bloc.dart + _event.dart + _state.dart (part of)
        ├── screens/   الشاشات
        └── widgets/   ويدجتس ونوافذ الميزة
```

- ميزة بلا خادم (شاشة "المزيد" مثلاً) تكتفي بـ `presentation/` .
- ملفات تختلف بين الويب والجهاز تُقسم بـ conditional export:

```
// cache_store.dart
export 'cache_store_io.dart' if (dart.library.js_interop) 'cache_store_web.dart';
```

6. الطبقات: من الـ API إلى الشاشة

```
Screen ──event──▶ Bloc ──▶ Repository (interface) ◀── implements ── RemoteDataSource ──▶
ApiClient ──▶ Dio
   ▲                |                                                          |
   └── BlocBuilder ◀┘◀──────────── ApiResult<T> (ApiSuccess | ApiFailure(Failure)) ◀──────┘
```

العقد والتنفيذ

```
abstract interface class ProfileRepository {
  Future<ApiResult<Profile>> get();
  Future<ApiResult<void>> update(Json body);
}

class ProfileRemoteDataSource implements ProfileRepository {
  ProfileRemoteDataSource(this._api);
  final ApiClient _api;
  static const _base = '/profile';

  @override
  Future<ApiResult<Profile>> get() => _api.getObject(_base, fromJson: Profile.fromJson);

  @override
  Future<ApiResult<void>> update(Json body) => _api.put(_base, body: body, parse: Parse.none);
}
```

الموديل

- كلاس عادي يمتد `Equatable` مع `factory fromJson(Json json)` مكتوب يدوياً، **بلا** code generation (لا freezed ولا json_serializable).
- القراءة من JSON آمنة عبر امتداد `obj` ، `date` ، `strList` ، `strOrNull` ، `str` ، `json.intv('id')` : `json_x` . غياب حقل أو تغيّر نوعه لا يرمي استثناء. `asList(raw, fromJson)`

حقن الاعتماديات (get_it)

النوع	التسجيل
التخزين، الاتصال، الكاش	`registerSingleton` (تُجهّز قبل runApp)
Dio، ApiClient، ThemeCubit، PushService	`registerLazySingleton`
المستودعات	`registerLazySingleton<XRepository>(() => XRemoteDataSource(sl()))`
حالة الجلسة (AuthBloc)، عدّاد الإشعارات، الشارات	`registerLazySingleton` بنسخة واحدة للتطبيق
Bloc الشاشة	`registerFactory` (نسخة لكل شاشة، تُغلق مع خروجها)

المتغير الوحيد: `final sl = GetIt.instance;` والتسجيل بدالتين: `configureDependencies()` للبنية و `registerFeatures(sl)` للميزات.

7. إدارة الحالة بلا setState

Slice: وحدة الحالة

```
enum LoadStatus { initial, loading, success, failure }

class Slice<T> extends Equatable {
  final LoadStatus status; final T? data; final Failure? failure;
  bool get isFirstLoad => (isLoading || isInitial) && data == null;   // هياكل رمادية
  bool get isRefreshing => isLoading && data != null;                 // المحتوى يبقى ظاهراً
  Slice<T> toLoading();  Slice<T> toSuccess(T v);  Slice<T> toFailure(Failure f);
  Slice<T> fromResult(ApiResult<T> r);  Slice<T> mapData(T Function(T) f);
}
```

كل حالة Bloc تتكون من شريحة أو أكثر (`Slice<Details> details` ، `Slice<Paged<Item>> list` ...). **البيانات السابقة تبقى أثناء إعادة التحميل.**

الأدوات الجاهزة

الأداة	متى
`LoadCubit<T>(fetch)`	قسم يقرأ قيمة واحدة: `refresh()` ، `load()` صامت، `patch()` تعديل محلي بعد الحفظ
`ListCubit<T>(fetch)`	قائمة موبايل: الصفحة الأولى، `more()` "عرض المزيد"، `filter(key, value)` يعيد من الأول
`ValueCubit<T>` / `ToggleCubit`	بديل setState لأي حالة واجهة محلية
`AppFormCubit`	أي نموذج أو تأكيد (القسم 14)
`Bloc` كامل	شاشة فيها أكثر من شريحة أو منطق: أحداث `final class` + `sealed` ، حالة بـ `copyWith`
`OnInit`	طلب أول تحميل من Bloc موجود أعلى الشجرة عند دخول الشاشة
`pullRefresh(bloc, trigger)`	السحب للتحديث: ينتظر حتى تهدأ حالات الـ Bloc
`Hover`	متابعة مرور الماوس وبناء العنصر حسبه، بلا setState

> الاستثناء الوحيد المسموح: `StatefulWidget` داخل ويدجتس البنية (OnInit، Hover، الجدول، Splash) لإدارة دورة الحياة والـ controllers، وليس لحالة الشاشة.

```
class AdminsBloc extends Bloc<AdminsEvent, AdminsState> {
  AdminsBloc(this._repo) : super(const AdminsState()) {
    on<AdminsListRequested>(_onList);
    on<AdminsListRefreshed>((e, emit) => _onList(AdminsListRequested(state.query), emit));
  }
  Future<void> _onList(AdminsListRequested e, Emitter<AdminsState> emit) async {
    emit(state.copyWith(query: e.query, list: state.list.toLoading()));
    emit(state.copyWith(list: state.list.fromResult(await _repo.list(e.query))));
  }
}
```

8. الشبكة والأخطاء

عقد الرد مع الباك اند

```
{ "success": true, "message": "...", "data": {...} | [...],
  "meta": { "pagination": {...}, "summary": {...} } }
// الخطأ: { "success": false, "code": "VALIDATION_ERROR", "message": "...", "errors": {
"owner.phone": ["..."] } }
```

Dio واحد بالتطبيق (DioFactory)

- `BaseOptions`: الرابط من `AppConfig`، المهلات، `Accept: application/json`، و `listFormat:` `ListFormat.multiCompatible` (مصفوفات لارافيل `ids[]=1`).
- ترتيب الـ interceptors (موبايل): **الكاش أولاً** ← فحص الاتصال ← التوكن. بالداشبورد: التوكن فقط.
- بالـ debug: سطر واحد لكل طلب `[API] GET /path -> 200`.
- الرابط يتغير دون تعديل الكود: `--dart-define=API_BASE_URL=https://.../api/v1`.

ApiClient: واجهة الطلبات الوحيدة

`get / post / put / patch / delete / upload` مع `parse`، ومختصرات: `getPaged`، `getList`، `getObject` (يرجع `Paged<T>` = عناصر + `meta` + `summary` + `Pagination`)، `getWithMeta`، و `postBytes` / `getBytes` للملفات (بـ `as=base64` حتى لا تلتقطها برامج التحميل بالمتصفح). محوّلات جاهزة: `Parse.intField`، `Parse.field`، `Parse.object`، `Parse.json`، `Parse.none`.

Failure و ApiResult

```
sealed class ApiResult<T> { dataOrNull; failureOrNull; when(success:, failure:); map(); }
final class ApiSuccess<T> extends ApiResult<T> { final T data; final String? message; }
final class ApiFailure<T> extends ApiResult<T> { final Failure failure; }
```

- `FailureType` يطابق حقل `code` الثابت بالباك (UNAUTHENTICATED، FORBIDDEN، NOT_FOUND، CONFLICT، VALIDATION_ERROR، TOO_MANY_REQUESTS، MAINTENANCE، SUBSCRIPTION_EXPIRED، PLAN_LIMIT_REACHED...) مع احتياط من رقم الحالة (401، 403، 404، 409، 422، 429، 503، 5xx) وأخطاء الشبكة المحلية (network، timeout، cancelled).

- `ErrorHandler.handle` يحوّل أي استثناء (Dio، Format، Type) لـ `Failure` برسالة عربية: ماذا حدث ثم ماذا تفعل.
- `validation_translator` يترجم رسائل التحقق الافتراضية الإنجليزية من لارافيل إلى عربية.
- `failure.fieldError('owner')` يدعم المفاتيح المتداخلة.
- `AuthInterceptor` : يضيف `Bearer` ، ويحوّل أخطاء الجلسة (401، حساب موقوف، تغيير كلمة سر إجباري) لأحداث `SessionEvent` يستمع لها `AuthBloc` فيعيد الراوتر التوجيه فوراً. أخطاء طلب الدخول نفسه تُعرض بالنموذج.
- `GlobalErrorGuard` : شبكة أمان لأي خطأ غير ملتقط (يُسجَّل، والمستخدم يرى رسالة عربية بدل شاشة رمادية). وإن فشل التشغيل نفسه تظهر شاشة "تعذّر تشغيل التطبيق" بدل شاشة بدء معلّقة.

9. الكاش والعمل دون اتصال

> **القرار:** الكاش على مستوى الـ HTTP (interceptor) وليس على مستوى الـ Bloc. لا `hydrated_bloc` . السبب: كل شاشة تعمل دون اتصال تلقائياً دون أي كود بالميزة، والكاش مربوط بالتوكن ويُمسح كاملاً عند الخروج. `hydrated_bloc` يبقى خياراً فقط لحفظ حالة واجهة بحتة (ليست بيانات خادم) إن احتجتها.

OfflineCacheInterceptor (للقراءة فقط)

1. يُخزَّن فقط: `GET` بـ JSON، ليس فيه `extra[skip]` ولا `as=base64` .
2. **المفتاح** = بصمة (FNV-1a) لـ `token | uri` : كل حساب له كاشه.
3. **بالاتصال:** يرسل `If-None-Match` بالـ ETag المحفوظ. رد 304 ← تُعرض النسخة المحفوظة دون تنزيل. رد 200 ← يُحفظ مع ETag ووقت الحفظ. ومع وجود نسخة احتياطية تقل مهلة الاتصال لـ 8 ثوانٍ.
4. **دون اتصال أو فشل شبكة:** تُعرض آخر نسخة فوراً كرد 200 مع `extra[fromCache]=true` ، ويُسجّل "آخر تحديث قبل..." للشريط العلوي.
5. **الكتابة تحتاج اتصالاً:** `ConnectivityInterceptor` يرفضها فوراً برسالة واضحة.

التخزين

- `CacheStore` (io) : ملف JSON لكل رد بمجلد التطبيق الخاص (`getApplicationSupportDirectory()/api_cache`)، حد 400 رد ويحذف الأقدم كل 40 كتابة، ولا يُحفظ رد أكبر من 1.5MB. أي خطأ قرص يُتجاهل (الكاش اختياري).
- `CacheStore` (web): `localStorage` للمعاينة فقط.
- يُمسح الكاش مع الجلسة (خروج، انتهاء، إيقاف حساب) عبر `SessionStorage(onClear: cache.clear)` .

ConnectionCubit: فحص الإنترنت الموحّد

- `connectivity_plus` يلتقط تغيّر الشبكة فوراً، و `internet_connection_checker_plus` يتأكد أن الإنترنت فعلي.
- يبدأ مع التطبيق ويعيش طوال عمره. أي فشل شبكة بطلب يطلب `recheck()` فوري.

- `OfflineBanner` فوق كل الشاشات (Stack بالـ builder العام)، و `offline_views` حالة "لا اتصال" موحّدة تعيد المحاولة وحدها عند عودة الاتصال.
- عودة الاتصال ← الشاشات تتحدث وحدها (`AppPage`) و `cache.markOnline()` .

تخزين الجلسة

- الموبايل: التوكن بـ (Keychain/Keystore) `flutter_secure_storage` ويُقرأ قبل runApp.
- الويب: `shared_preferences` .
- التفضيلات (الثيم...) بـ `shared_preferences` .

10. الراوتر والتنقل

- `go_router` بـ `refreshListenable` من stream الـ `AuthBloc` ، و `redirect` حسب `unknown` : `AuthStatus` ، `authenticated` ، `passwordChangeRequired` ، `offline` ، `unauthenticated ← login` ، `splash ←` .
- الصفحة المطلوبة تُحفظ بـ `?from=` أثناء التحقق أو الدخول ويعود لها المستخدم بعده.
- `Routes` : ثوابت + دوال للمسارات ذات المعرّف (`Routes.isoDate` + (`Routes.child(id)`) .
- الداشبورد: `ShellRoute` للهيكل، والمسارات مقسمة بملفات حسب المجال (`routes_management.dart` ، `routes_operations.dart` ...) حتى لا يتجاوز ملف 300 سطر.
- `blocRoute<B>(path, create:, child:)` : صفحة مع Bloc خاص يُنشأ عند فتحها ويُغلق عند مغادرتها. مع `idOf(state)` و `paramOf(state, name)` .
- الموبايل: تبويبات (`Routes.tabs`) وصفحات تُفتح فوقها. `context.open(location)` يبدّل التبويب أو يدفع صفحة، و `context.back()` يرجع أو يذهب للرئيسية إن فُتحت الصفحة مباشرة (من إشعار).
- انتقال الشاشات العامة بـ `fadePage` (تلاشٍ بمدة `AppMotion.slow`).

11. الداشبورد (Flutter Web)

- **AppShell:** `Sidebar` + `Topbar` + المحتوى. موبايل: درج بعرض 280. تابلت: مطوية. كمبيوتر: مفتوحة وتُطوى بزر (حالتها بـ `ToggleCubit`). عناصر القائمة من `nav_config.dart` .
- **التجاوب:** `context.responsive(mobile:, tablet:,` و `Breakpoints.mobile = 600` ، `tablet = 1024` ، `controlHeight` ، `cardPadding` ، `context.pagePadding` ، `desktop:)` و `ResponsiveGrid(minItemWidth: 240, maxColumns: 4)` .
- **الجداول:** `PagedTable<T>` = SliceView + `TableShimmer` + `AppDataTable` + `PaginationBar` . أي تغيير (صفحة، ترتيب، مسح فلاتر) يخرج بـ `onQuery(ListQuery)` ليطلبه الـ Bloc. فارغ مع فلاتر = "لا نتائج" بزر مسح. `AppDataTable` كامل على الكمبيوتر، تمرير أفقي على التابلت، وبطاقات على الموبايل.
- **ListQuery:** الصفحة، البحث، الترتيب، الفلاتر، `hasFilters` ، `cleared()` ، `withFilter` .
- **FilterBar:** بحث بـ `Debouncer` + فلاتر + مسح، يلتف على الشاشات الضيقة.
- **الإجراءات:** `RowActions` (قائمة منسدلة بأيقونات ملوّنة)، `DetailDialog` (يجلب العنصر عند الفتح)، `FormDialog` لكل تعديل، `CredentialsDialog` (بيانات دخول تُعرض مرة واحدة)، `ExportDialog` .

- **المكونات:** `StatCard` (أيقونة + رقم يعدّ تصاعدياً)، `HeroBanner` (المكان المزخرف)، `InfoGrid`، `AppTabs`، `StatusBadge`، `ChartLegend` (الرسوم `fl_chart` فوق `BarsChart`، `TrendChart`).
- البحث الشامل والتنبيهات تفتح الصفحات بفلتر أولي عبر query parameters.
- التشغيل: `flutter run -d chrome`، والبناء: `flutter build web --release --dart-define=API_BASE_URL=...`.

12. تطبيقات الموبايل

- **main:** `SystemUiMode.edgeToEdge` ← `GlobalErrorGuard.install()` ← `PushService.init()` ← `configureDependencies()` (لا يوقف التشغيل إن فشل) ← runApp. أي فشل = الاتجاه عمودي فقط ← شاشة خطأ تشغيل واضحة.
- **app.dart builder:** `MediaQuery.withClampedTextScaling` ← `AnnotatedRegion<SystemUiOverlayStyle>` ← `Stack[SessionScope(child), OfflineBanner]` ← إخفاء الـ scrollbars. و `scaffoldMessengerKey` عام لعرض الرسائل من أي مكان.
- **MobileShell:** أربعة تبويبات كل منها يحفظ حالته، وشريط سفلي عائم (`FancyTabBar`) بزر أوسط يفتح قوس إجراءات سريعة (`QuickActions`)، وشارات أعداد على التبويبات (`NavBadgesCubit`). الرجوع من تبويب غير الرئيسية يعود للرئيسية.
- **SessionScope:** حالة الجلسة المشتركة فوق الراوتر (عدّاد الإشعارات، الشارات، خيارات مشتركة) تبدأ عند الدخول وتُمسح عند الخروج.
- **AppPage + MobileHeader:** هيكل أي شاشة: رأس بلون الهيكل (عنوان، وصف، رجوع تلقائي، زخرفة) ثم الأزرار ثم المحتوى، والسحب للتحديث.
- **القوائم:** `LoadMoreList` فوق `ListCubit`، و `ActionSheet` لإجراءات العنصر.
- **الإشعارات:** `flutter_local_notifications` + `firebase_messaging`. دورة التوكن: إذن ← تسجيل التوكن بالخادم عند الدخول وعودة الاتصال وتجدده ← حذفه عند الخروج. `onOpen` يفتح صفحة الإشعارات (بعد انتهاء البدء)، و `onMessage` يحدّث العدّاد. `push_permission` + `app_settings` لتنبيه "الإشعارات مطفية" وفتح الإعدادات.
- **الملفات:** `BlobOpener`: على الجهاز يُجلب الملف المحمي ويُحفظ (`path_provider`) ويُفتح (`open_filex`)، وعلى الويب blob. رفع الملفات بـ `file_picker` و `FormData`.
- **HTML من الخادم** (معاينة شهادة/فاتورة): `webview_flutter` على الجهاز و iframe على الويب.
- **اختيار التطبيق المسموح للدور:** طلب الدخول يرسل اسم التطبيق، والخادم يرفض الدور غير المسموح.
- البناء: `flutter build apk --release`، ومحلياً للمعاينة: `flutter build web --debug --dart-define=API_BASE_URL=http://127.0.0.1:8000/api/v1`.

13. التحميل والحالات الأربع

```
SliceView<T>(
  slice: state.details,
  loading: const DetailsShimmer(),            // هيكل بنفس شكل المحتوى
  onRetry: () => bloc.add(const DetailsRequested()),
  isEmpty: (d) => d.items.isEmpty,
  empty: EmptyState(...),                      // دعوة لا اعتذار
  builder: (context, data) => DetailsView(data),
)
```

- **أول تحميل:** هياكل رمادية (`shimmer`) **مطابقة لشكل المحتوى**: `AppShimmer` يحرك كل `ShimmerBox` بداخله، بلون `surfaceAlt` ، والبطاقة الهيكلية حدّ فقط بلا تعبئة. جاهز: `TableShimmer` ، هياكل البطاقات والقوائم والإحصاءات. لا `CircularProgressIndicator` بوسط الشاشة.
- **إعادة التحميل:** المحتوى يبقى بشفافية 0.55 + شريط تقدّم رفيع 2px بالأعلى، والضغط معطّل مؤقتاً.
- **خطأ بلا بيانات:** `ErrorState(failure, onRetry)` (وحالة لا اتصال الموحّدة إن كان السبب الشبكة). خطأ مع بيانات سابقة: تبقى البيانات.
- **التبديل بين الحالات:** `AnimatedSwitcher` بمدة `normal` ، والمحتوى يدخل بـ `flutter_animate` (`fadeIn`).
- `frame` اختياري حول حالات التحميل والخطأ (ليبقى رأس الصفحة وزر الرجوع ظاهرين).

14. النماذج والتحقق

- **أسماء الحقول بصيغة لارافيل المنقّطة** (`owner.phone`) لتطابق مفاتيح أخطاء 422 مباشرة، و `apiBody()` / `form.payload` يحوّلها لشكل متداخل.
- `AppFormCubit` : النصوص بـ controllers تُنشأ عند الطلب (`form.text(name)`) وتُتلف معه، وغير النصية بـ `set(name, value)` . `submit(send)` : تحقق محلي ← إرسال ← توزيع أخطاء 422 على الحقول، وأخطاء حقول غير ظاهرة تُعرض أعلى النموذج (`orphanErrors`)، والفشل العام (409، شبكة) أعلاه.
- `apiBody(keepNulls:, dateTimes:, except:)` : التواريخ `2026-10-07` أو ISO كامل لحقول محددة، واستبعاد حقول.
- الحقول المربوطة: `FormPassword` ، `FormText` (إظهار/إخفاء بـ Cubit)، `FormDropdown` ، منتقي التاريخ، الاختيار المتعدد برقائق، `FormFile` ، داخل `FormColumn` . خيارات القوائم من الخادم بـ `OptionsLoader` .
- `Validators` : رسائل تقول المطلوب مباشرة (`newPassword()` ، `email()` ، `required('اكتب اسمك.')` ...).

```
final done = await FormDialog.show<void>(
  context,
  title: 'تعديل بياناتي', icon: Icons.person_rounded,
  initial: {'name': profile.name, 'email': profile.email},
  fields: (_) => FormColumn(children: [
    FormText('name', label: 'الاسم', required: true, validator: Validators.required('اكتب
اسمك.')),
    FormText('email', label: 'البريد الإلكتروني', ltr: true, validator: Validators.email()),
  ]),
  onSubmit: (form) => sl<ProfileRepository>().update(form.apiBody(keepNulls: true)),
);
if (done != null && context.mounted) context.read<ProfileBloc>().add(const
ProfileRequested(silent: true));
```

نوافذ التأكيد (حذف، إيقاف، خروج من كل الأجهزة) هي نفس `FormDialog` بلا حقول مع `danger: true`. رسالة النجاح بـ `AppSnackbar`.

15. الباكجات المعتمدة

الباكج	الدور	أين
`flutter_bloc` + `equatable`	إدارة الحالة ومقارنة الحالات	الكل
`get_it`	حقن الاعتماديات	الكل
`go_router`	التوجيه والحماية بالمصادقة	الكل
`dio`	الطلبات والـ interceptors	الكل
`shared_preferences`	التفضيلات وجلسة الويب	الكل
`flutter_secure_storage`	التوكن الآمن	الموبايل
`shimmer`	هياكل التحميل	الكل
`flutter_animate`	حركات الدخول والتلاشي	الكل
`fl_chart`	الرسوم البيانية	الكل
`flutter_svg`	الشعار والأيقونات SVG	الكل + حزمة التصميم
`intl` + `flutter_localizations`	العربية والتواريخ	الكل
`file_picker`	اختيار ملفات للرفع	الكل
`url_launcher`	الروابط الخارجية	الكل
`web`	blob والتنزيل على الويب	الكل

`path_provider` + `open_filex`	حفظ الملفات وفتحها وكاش الردود	الموبايل
`connectivity_plus` + `internet_connection_checker_plus`	فحص الاتصال الموحّد	الموبايل
`webview_flutter`	عرض صفحات HTML من الخادم	الموبايل
`firebase_core` + `firebase_messaging` + `flutter_local_notifications`	إشعارات الجهاز	الموبايل
`app_settings`	فتح إعدادات الإشعارات بالجهاز	الموبايل
`flutter_launcher_icons` / `flutter_native_splash`	الأيقونة وشاشة البدء الأصلية	الموبايل (dev)
`flutter_lints`	قواعد التحليل	الكل (dev)

غير مستخدم عمداً: `hydrated_bloc` (الكاش بمستوى HTTP)، `freezed` / `json_serializable` (موديلات يدوية بـ json_x)، `retrofit` (ApiClient واحد)، `provider` / `getx` / `riverpod` .

> ملاحظة: إن ظهر تعارض نسخ بالأندرويد ثبّت النسخة بـ `dependency_overrides` (مثل `shared_preferences_android`).

16. التسمية والكود والـ Lint

- الملفات `snake_case` بلاحقة دورها: `_repository` ، `_cubit` ، `_state` ، `_event` ، `_bloc` ، `_screen` ، `_remote_datasource` ، `_models` .
- الأحداث بصيغة الماضي: `XSubmitted` ، `XRefreshed` ، `XRequested` .
- الكلاسات المساعدة الثابتة (Routes، AppSpace، ErrorHandler، Parse) `abstract final class` .
- تعليقات التوثيق `///` بالعربية بأعلى كل كلاس: ماذا يفعل ولماذا، وليس كيف.
- `Labels` : ترجمة قيم الـ API الثابتة (الحالات والأنواع) لنص عربي + لون + أيقونة بمكان واحد، والمصطلحات ثابتة بكل الواجهات.
- `Formatters` : الأرقام لاتينية، والتواريخ `2026-10-07` .
- بعد أي `await` بالشاشة: `if (!context.mounted) return;` ، وبالـ Cubit: `if (isClosed) return;`.

```
include: package:flutter_lints/flutter.yaml
formatter:
  page_width: 120
analyzer:
  exclude: [build/**, web/**, android/**, ios/**]
```

17. قائمة البدء

مشروع جديد

1. الهوية أولاً: الفكرة + أربع قواعد ← `tokens.json` ← الشعار بنسخه ← الخطوط ← `brand-` ← `identity-spec.md` ← `guide.html`.
2. حزمة (tokens، theme، motion، ornaments، widgets) `<brand>_design` مع تطبيق عرض `example/` .
3. `flutter create` بمسار إنجليزي، ثم نسخ `core/` كاملاً من آخر مشروع (يُعدّل `app_config` و `routes` و `labels` فقط).
4. إضافة الحزمة كـ path dependency، و `analysis_options.yaml` بالإعداد أعلاه.
5. الاتفاق مع الباك اند على: غلاف الرد، أكواد الأخطاء، شكل ETag/304 ، `meta.pagination` لطلبات القراءة، واسم التطبيق بطلب الدخول.
6. ميزة `auth` أولاً (splash، login، change password، offline) ثم الهيكل (Shell) ثم الميزات.
7. الموبايل: Firebase، الأيقونة، شاشة البدء، الأذونات.

ميزة جديدة

1. `domain/models/x_models.dart` : الموديلات بـ `fromJson` عبر json_x.
2. `data/repo/x_repository.dart` : العقد.
3. `data/datasources/x_remote_datasource.dart` : التنفيذ عبر `ApiClient` .
4. التسجيل بـ `feature_injection.dart` (المستودع lazySingleton، الـ Bloc factory).
5. Bloc : `presentation/bloc/` بشرائح `Slice` (أو `ListCubit` / `LoadCubit` إن كفى).
6. المسار بـ `blocRoute` + `Routes` .
7. الشاشة: `PagedTable` / `SliceView` + `AppPage` بحالاتها الأربع، والهيكل الرمادي بشكل المحتوى.
8. التعديلات بـ `FormDialog` ، والتحديث بعدها صامت.
9. مراجعة: لا setState، لا ملف فوق 300 سطر، لا قيمة تصميم أو مسار مكتوب، الثيم الداكن، شاشة 360px بلا overflow، و `flutter analyze` نظيف.